

高级语言程序设计

实验报告

题目：基于 C++ 与 YOLOv8 的美团单车目标检测系统

南开大学 软件学院

姓名：文旺

学号：2410452

班级：软件二班

指导教师：郭文雅

完成日期：2026 年 5 月 5 日

目录

高级语言程序设计大作业实验报告	1
一、作业题目	1
二、开发环境与开发软件	1
三、课题背景与需求分析	2
四、系统总体设计	3
1. 整体流程	3
2. 模块划分	5
3. 数据流设计	6
五、核心算法与实现方法	6
1. 图像预处理	6
2. ONNX Runtime 模型推理	6
3. YOLO 后处理与 NMS	7
4. Qt 可视化界面设计	7
六、数据采集、标注与模型训练	8
七、系统测试与运行结果	11
八、项目版本演进	14
九、问题分析与改进方向	15
十、实验总结	16

高级语言程序设计大作业实验报告

一、作业题目

本次大作业题目为：

基于 C++ 与 YOLOv8 的美团单车目标检测系统

本项目围绕校园共享单车管理场景，设计并实现了一个面向美团单车识别的桌面端目标检测系统。系统使用 C++ 作为主要开发语言，结合 Qt Widgets 构建图形化界面，使用 OpenCV 完成图像读取与预处理，使用 ONNX Runtime 部署 YOLOv8 模型，实现对图片中美团单车目标的检测、定位与可视化展示。

项目最终形成一个可运行的桌面应用，用户可以通过界面选择本地图片，点击检测按钮后，系统自动完成模型推理，并在图像上绘制检测框、类别标签和置信度信息。同时，界面左侧提供检测任务选择、置信度阈值控制和结果统计区域，为后续扩展多目标检测任务预留了接口。

二、开发环境与开发软件

2.1 开发环境

本项目主要在 Windows 平台下完成开发、构建与测试。

项目	内容
操作系统	Windows 10 / Windows 11
编程语言	C++17
IDE	Visual Studio / Qt Creator / Visual Studio Code (TODO: 按实际填写)
构建工具	CMake + Ninja
编译器	MSVC 2022 x64
图形界面库	Qt Widgets
图像处理库	OpenCV
模型推理库	ONNX Runtime
深度学习模型	YOLOv8n 自训练模型

项目	内容
数据标注平台	Roboflow
模型训练平台	Google Colab

2.2 第三方库说明

- 1. Qt Widgets
用于构建桌面图形界面，包括主窗口、按钮、下拉框、滑块、文本结果面板以及图像显示区域。
- 2. OpenCV
用于读取本地图片、进行图像预处理，并辅助完成 YOLO 输入张量构造。
- 3. ONNX Runtime
用于在 C++ 环境中加载并运行导出的 best.onnx 模型，实现脱离 Python 环境的本地推理。
- 4. YOLOv8 / Ultralytics
用于模型训练和导出。本项目采用 yolov8n.pt 作为迁移学习基础模型，在自采集美团单车数据集上训练得到 best.onnx。

三、课题背景与需求分析

3.1 课题背景

共享单车在校园中使用频率较高，但也常出现车辆堆放杂乱、停放位置不规范等问题。若要进一步实现共享单车数量统计、区域监管或违停识别，首先需要让计算机能够从图像中准确识别出共享单车的位置。

传统图像处理方法通常依赖颜色、边缘或形状特征，在复杂背景、遮挡、光照变化和多目标重叠情况下鲁棒性较差。相比之下，基于深度学习的目标检测模型能够从大量样本中学习目标的视觉特征，更适合处理真实校园环境中的复杂图像。

因此，本项目选择 YOLOv8 作为目标检测模型基础，结合 C++ 工程部署能力，实现一个可运行、可展示、可扩展的美团单车目标检测系统。

3.2 需求分析

项目需要实现以下基本需求：

- 1. 图片读取
用户可以通过图形界面选择本地图片，程序能够读取并显示该图片。
- 2. 模型加载
程序能够加载训练好的 ONNX 模型文件 best.onnx。
- 3. 目标检测

程序能够对输入图片执行 YOLO 推理，得到目标位置、类别和置信度。

4. 后处理

程序能够完成置信度过滤、坐标还原和非极大值抑制，得到最终检测结果。

5. 可视化展示

程序能够在界面中绘制检测框和标签，并显示检测数量及置信度列表。

6. 交互控制

用户可以通过置信度滑块动态筛选检测结果。

7. 可扩展性

当前版本主要检测美团单车，但界面和结构需要为后续扩展其他检测任务预留空间。

四、系统总体设计

4.1 设计思路

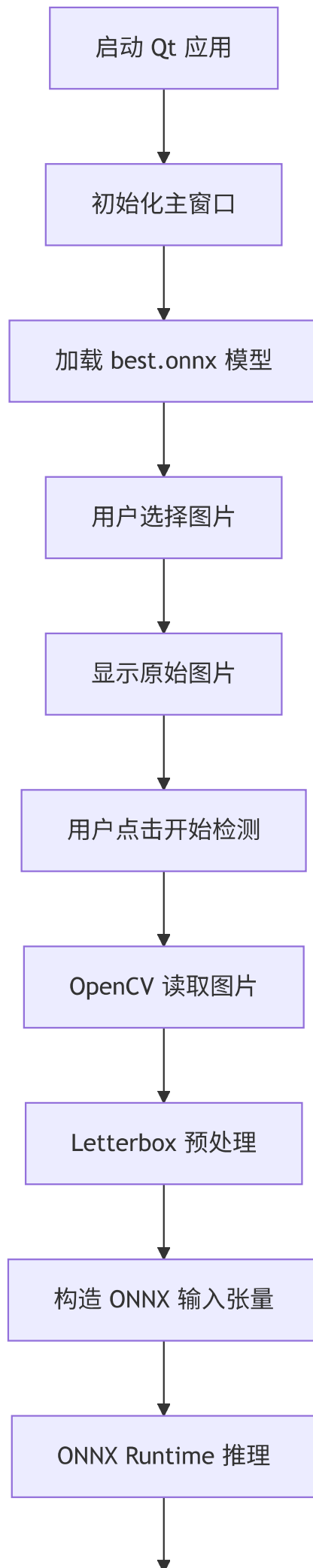
本项目的整体设计思路是将目标检测系统拆分为“界面交互层”“图像处理与推理层”“结果可视化层”三个部分。

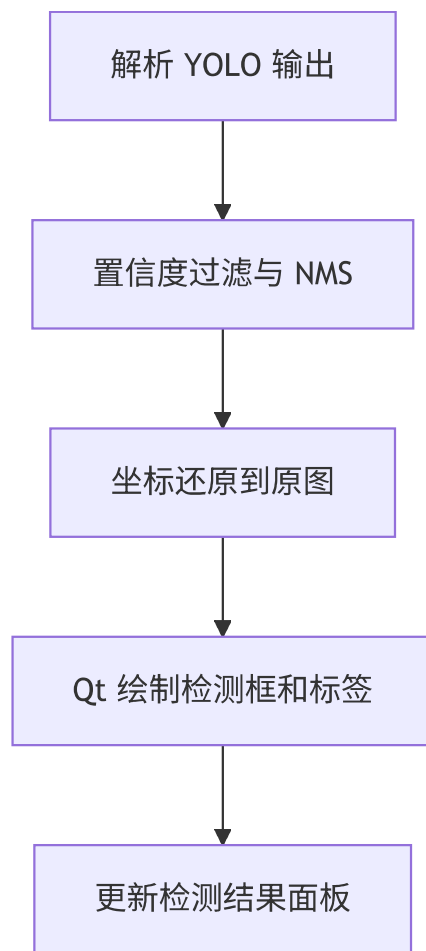
界面交互层负责接收用户操作，例如选择图片、点击检测按钮、调整置信度阈值等；图像处理与推理层负责将图片转换为模型输入，调用 ONNX Runtime 完成推理，并对输出张量进行解析；结果可视化层负责将检测结果映射回原图，并在 Qt 画布中进行展示。

这样设计的好处是系统逻辑清晰，各模块职责相对独立，便于后续维护和扩展。例如，如果之后需要加入“电动车检测”“行人检测”或“违停区域判断”，可以在模型配置和任务选择模块上扩展，而不必完全重写界面和推理流程。

4.2 整体流程

系统整体流程如下：





4.3 模块划分

项目主要模块如下：

模块	对应文件	功能
程序入口	src/main.cpp	创建 Qt 应用并显示主窗口
主窗口模块	src/mainwindow.cpp/h/ui	负责 UI 交互、图片显示、检测触发和结果展示
检测器模块	src/yolo_detector.cpp 、 include/yolo_detector.h	负责模型加载、预处理、推理和后处理
数据结构模块	include/types.h	定义检测结果结构体
模型文件	models/best.onnx	美团单车检测模型
测试数据	data/	本地测试图片

4.4 数据流设计

用户选择图片后，系统首先保存图片路径，并使用 `QPixmap` 在界面中显示。点击“开始检测”后，程序使用 OpenCV 的 `cv::imread` 读取同一张图片，然后将其传入 `YoloDetector::process`。

检测器内部先对图片进行 Letterbox 预处理，将不同尺寸的图片统一变换为 `640 x 640` 的模型输入；随后使用 ONNX Runtime 执行模型推理，得到 YOLO 输出张量；最后通过后处理函数筛选出最终检测结果，并返回 `std::vector<Detection>`。

主窗口接收到检测结果后，一方面在 `QGraphicsScene` 中绘制检测框和标签，另一方面在左侧结果面板中显示检测数量和各目标置信度。

五、核心算法与实现方法

5.1 图像预处理

目标检测模型通常要求固定尺寸的输入。本项目使用的 YOLOv8 ONNX 模型输入尺寸为：

`1 x 3 x 640 x 640`

为了保持原图比例，项目采用 Letterbox 预处理方法，而不是直接强行拉伸图片。

Letterbox 的基本步骤如下：

1. 根据原图宽高和目标尺寸 `640 x 640` 计算缩放比例。
2. 按等比例缩放原图，使其完整放入目标尺寸中。
3. 对剩余区域填充灰色边框，使最终图像尺寸达到 `640 x 640`。
4. 将 BGR 图像转换为 RGB。
5. 将像素值归一化到 `[0, 1]`。
6. 将图像从 HWC 格式转换为 CHW 格式。

这样可以避免单车形状被强制拉伸变形，从而提高模型推理稳定性。

5.2 ONNX Runtime 模型推理

项目使用 ONNX Runtime 在 C++ 中加载并执行 `best.onnx` 模型。

推理流程主要包括：

1. 创建 `Ort::Env` 环境。
2. 创建 `Ort::SessionOptions` 并设置线程数。
3. 加载 ONNX 模型文件。
4. 创建输入张量。

5. 指定输入节点名和输出节点名。
6. 调用 `session->Run` 执行推理。
7. 获取输出张量数据指针。

当前模型输入节点名为：

`images`

输出节点名为：

`output0`

输出张量形状为：

`1 x 6 x 8400`

其中 6 表示每个预测框包含 4 个坐标信息和 2 个类别相关分数。当前模型主要使用 `shared_bike` 类别完成美团单车检测。

5.3 YOLO 后处理与 NMS

YOLO 模型原始输出包含大量候选框，不能直接作为最终结果使用。本项目后处理流程如下：

1. 遍历 8400 个候选框。
2. 读取每个候选框的中心点坐标、宽高和类别置信度。
3. 选出置信度最高的类别。
4. 根据置信度阈值过滤低质量预测框。
5. 根据 Letterbox 缩放比例和填充偏移，将坐标还原到原图尺寸。
6. 对边界框进行边界保护，避免坐标越界。
7. 使用 OpenCV 的 `cv::dnn::NMSBoxes` 执行非极大值抑制。
8. 返回最终检测结果。

NMS 的作用是去除多个高度重叠的重复检测框，只保留置信度最高的结果。

5.4 Qt 可视化界面设计

界面采用左侧工作台和右侧检测画布布局。

左侧工作台包含：

- 系统标题：Meituan Vision
- 检测任务选择框
- 图片选择按钮

- 开始检测按钮
- 置信度阈值滑块
- 当前状态提示
- 检测结果概要和明细

右侧检测画布包含：

- 检测画布标题
- 模型标识 `best.onnx`
- 图片显示区域
- 检测框与目标标签

为了提升界面观感，项目对 Qt 默认控件进行了样式优化，包括深色主题、圆角面板、美团黄色按钮、自定义置信度滑块以及动态检测标签。

5.5 检测标签优化

在最初版本中，检测框和标签会随着图片缩放而出现过大或过小的问题。为了解决该问题，项目对检测标签进行了优化：

1. 根据当前 `QGraphicsView` 缩放比例计算目标框在屏幕中的显示尺寸。
2. 根据目标框显示大小动态调整标签字号。
3. 根据目标框大小动态调整检测框线宽。
4. 标签优先贴合检测框上边缘显示。
5. 当目标框靠近图片顶部时，标签自动移动到框内，避免显示在画布外。
6. 置信度以百分比形式展示，例如 美团单车 82%。

此外，置信度滑块使用 `QProxyStyle` 和 `QPainter` 自定义绘制圆形手柄，避免 Qt 样式表在 Windows 下将手柄渲染为方块的问题。

六、数据采集、标注与模型训练

6.1 数据采集

本项目的模型训练数据由本人在校园真实环境中拍摄。拍摄对象主要为校园内出现的美团单车，拍摄场景尽量覆盖不同光照、角度、距离、遮挡和背景复杂度，以提高模型在真实环境中的泛化能力。

数据采集时主要考虑以下场景：

1. 教学楼、图书馆、食堂、宿舍楼等人员密集区域。
2. 道路边、人行道、绿化带、停车区域等单车常见停放位置。
3. 多辆车密集停放或相互遮挡的复杂场景。
4. 单辆车近景、中景和远景。

5. 含普通自行车、道路、建筑、草地等的负样本场景。

实际使用数据规模：

总图片数：153 张

训练集：107 张，其中 11 张为背景图

验证集：31 张，其中 6 张为背景图

测试集：15 张

验证集目标实例数：157

6.2 数据标注

数据标注在 Roboflow 平台完成，标注类型为目标检测中的 Bounding Box 矩形框。

标注原则如下：

1. 每一辆可辨认的美团单车都单独绘制检测框。
2. 多辆单车重叠时，各自独立标注，允许框与框之间重叠。
3. 被遮挡或被画面裁切的单车，只标注画面中可见部分。
4. 普通私人自行车、摩托车、电动车等不作为目标类别标注。
5. 不包含美团单车的图片保留为空标注，用作负样本。
6. 标注框尽量贴合目标可见区域的上下左右边界。

当前主要类别为：

```
names:  
  0: shared_bike
```

在程序界面中该类别被显示为：

美团单车

6.3 模型训练

训练阶段使用 Google Colab 提供的 Tesla T4 GPU。训练代码基于 Ultralytics YOLOv8 接口，使用 `yolo8n.pt` 作为预训练模型进行迁移学习。

核心训练代码如下：

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

results = model.train(
    data=f"{dataset.location}/data.yaml",
    epochs=50,
    imgsz=640,
    batch=16,
    lr0=0.01,
    plots=True
)
```

训练配置如下：

参数	含义	当前设置
模型基座	迁移学习起点	yolov8n.pt
训练轮数	数据集重复训练次数	50
输入尺寸	模型输入图片大小	640
Batch Size	单次训练图片数量	16
优化器	参数优化方法	AdamW
实际学习率	训练日志显示值	0.001667
训练设备	Colab GPU	Tesla T4
训练耗时	训练日志显示	约 0.035 小时

6.4 模型导出

训练完成后，选择验证集上表现最好的 `best.pt`，并导出为 ONNX 格式：

```
from ultralytics import YOLO

model = YOLO("runs/detect/train/weights/best.pt")
model.export(format="onnx", dynamic=False)
```

导出后得到 `best.onnx`，并将其放入项目目录：

```
YOLODeploy/models/best.onnx
```

ONNX 导出信息如下：

输入形状：1 x 3 x 640 x 640
输出形状：1 x 6 x 8400
文件大小：约 11.7 MB

6.5 模型训练结果

最终验证结果如下：

指标	数值
Precision	0.562
Recall	0.554
mAP50	0.571
mAP50-95	0.302

类别 shared_bike 的验证结果如下：

Images: 25
Instances: 157
Precision: 0.562
Recall: 0.554
mAP50: 0.571
mAP50-95: 0.302

从训练日志来看，模型在训练前期 mAP 较低，随着训练轮数增加逐步收敛，后期 mAP50 稳定在约 0.57 左右。由于数据量仍然较小，且校园场景中存在遮挡、多目标重叠、背景复杂等问题，因此模型仍有继续优化空间。

七、系统测试与运行结果

7.1 测试方法

系统测试主要从以下几个方面进行：

1. 模型加载测试

检查程序启动后能否正确加载 best.onnx 模型。

2. 图片读取测试

选择不同格式、不同路径的图片，检查程序是否能够正常显示。

3. 检测结果测试

使用包含单辆、多辆、远距离和遮挡目标的图片进行检测，观察检测框位置和置信度。

4. 负样本测试

使用不包含美团单车的图片进行测试，观察是否出现误检。

5. 界面交互测试

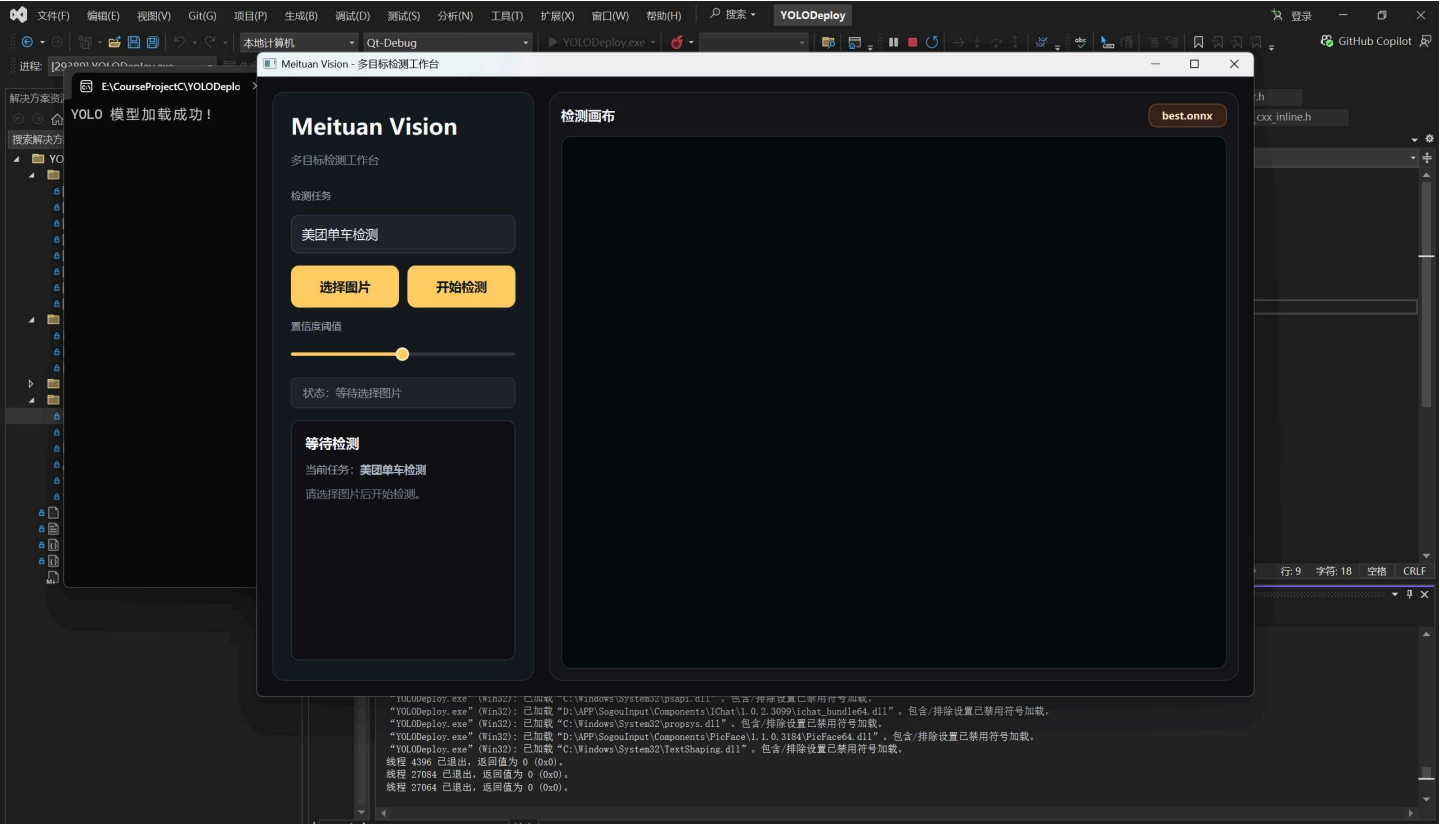
调整置信度滑块，观察检测框和结果列表是否同步更新。

6. 可视化效果测试

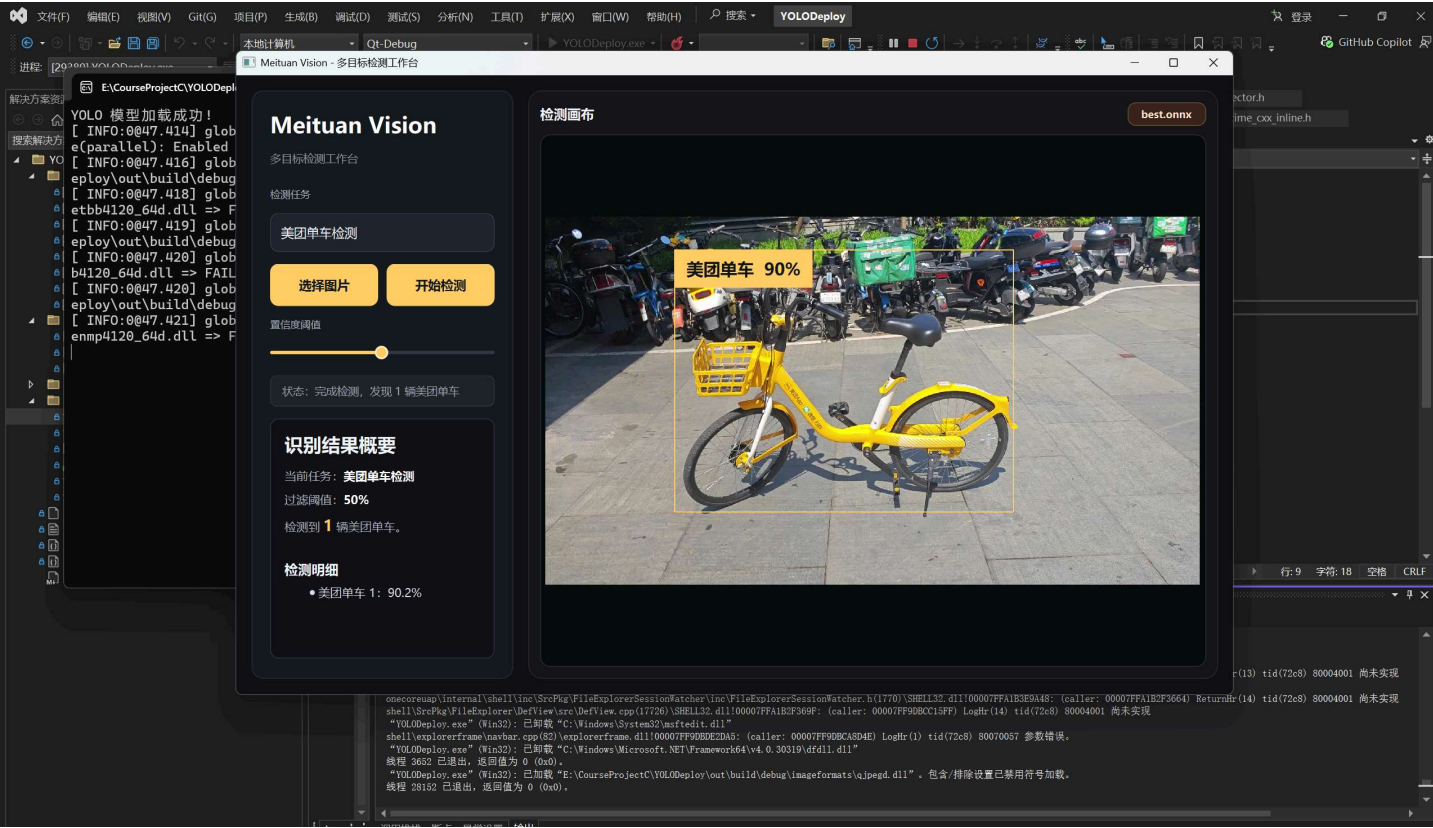
检查检测框、标签、字体大小和界面布局是否在不同分辨率图片下保持良好显示效果。

7.2 测试结果截图

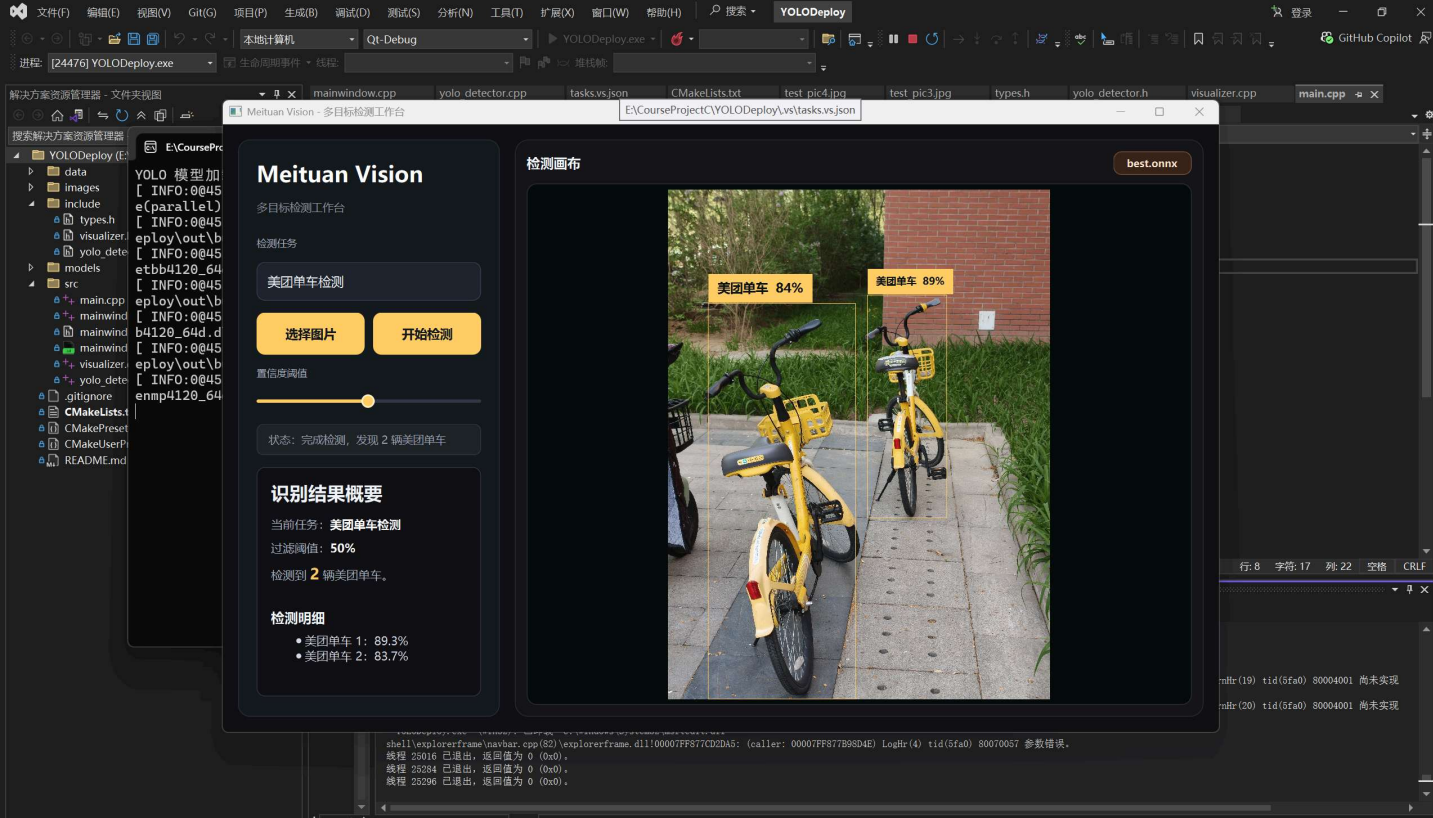
系统主界面:



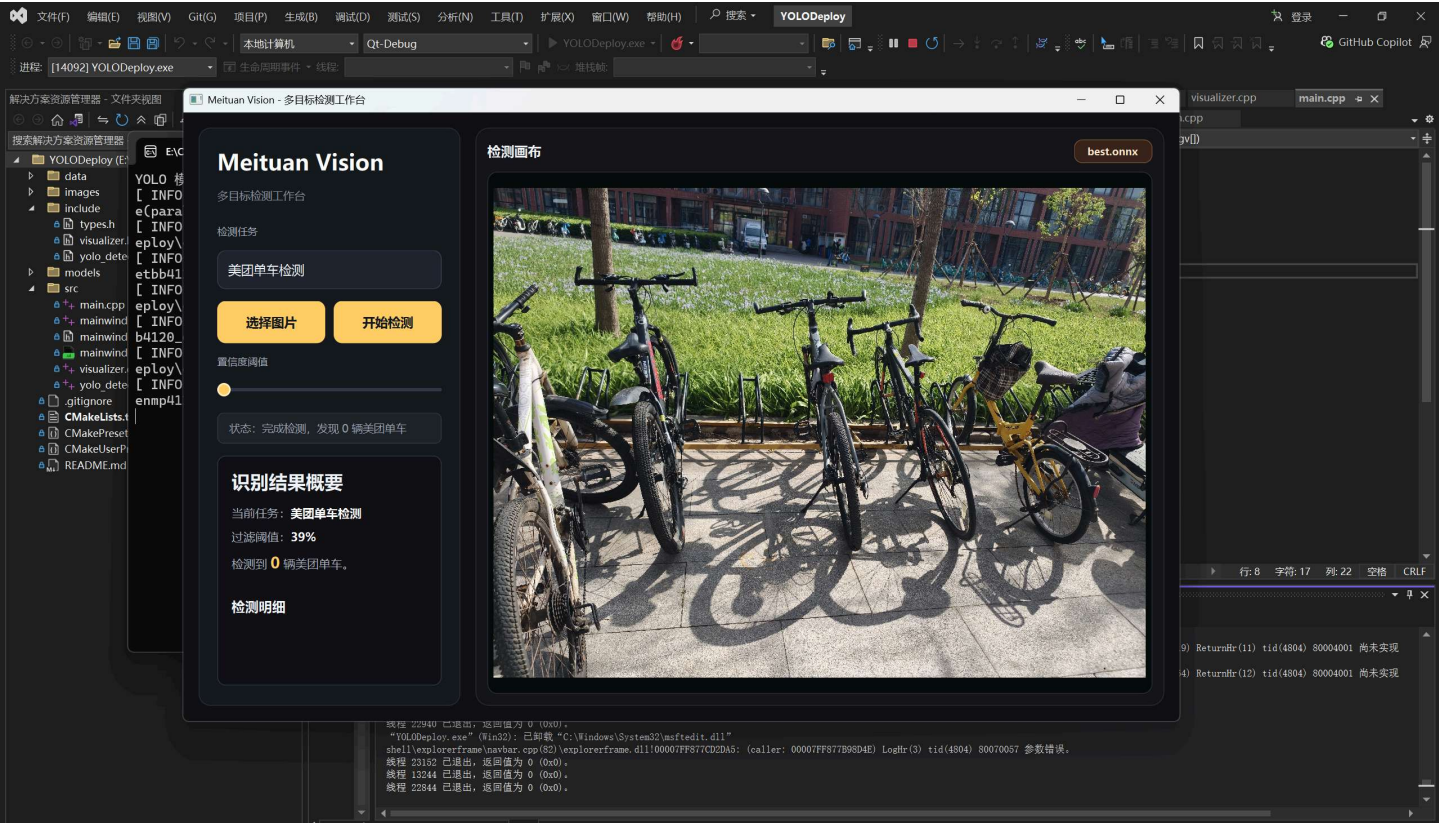
单目标检测:



多目标检测:



负样本检测:



7.3 结果分析

从当前测试结果来看，系统能够较好地完成美团单车检测任务。在目标清晰、距离适中、遮挡较少的场景中，模型能够给出较稳定的检测框和较高的置信度。在多目标密集场景中，模型能够检测出多辆单车，但对于远处小目标、严重遮挡目标或与背景颜色相近的目标，仍可能出现漏检或定位不够准确的情况。

界面方面，当前版本已经实现较完整的检测工作台效果。用户可以通过左侧面板完成任务选择、图片上传、检测启动和置信度调节；右侧画布能够展示检测图像和结果框。相比早期命令行或简单窗口版本，当前界面更适合课程展示和后续功能扩展。

八、项目版本演进

8.1 v1.0：YOLO 基础部署

v1.0 主要完成 C++ 目标检测部署基础能力：

- 搭建 C++ 部署环境。
- 集成 OpenCV 与 ONNX Runtime。
- 部署 YOLO 官方轻量级模型 `yo1ov8n.onnx`。
- 实现 Letterbox 图像预处理。
- 实现 YOLO 输出解析和 NMS 后处理。
- 支持 COCO 80 类目标检测与可视化画框。

8.2 v2.0：Qt 图形界面与专项模型

v2.0 在基础部署能力上加入图形化界面和美团单车专项模型：

- 集成 Qt Widgets 用户界面。
- 支持上传图片和展示检测结果。
- 使用自采集数据训练得到 `best.onnx`。
- 将检测目标从通用 COCO 类别转向美团单车专项识别。
- 支持置信度信息和检测数量展示。

8.3 v3.0：检测工作台界面优化

v3.0 主要优化界面体验和检测结果展示：

- 重构界面为左侧任务面板和右侧检测画布。
- 将标签统一为“美团单车”。
- 新增目标任务下拉框，预留多目标扩展入口。
- 优化深色主题、按钮、状态栏和结果面板。
- 优化检测框标签位置、字号和线宽。
- 自定义绘制圆形置信度滑块，修复 Qt 默认样式问题。

九、问题分析与改进方向

9.1 当前存在的问题

1. 路径硬编码问题

当前 `CMakeLists.txt` 和模型加载路径中仍存在本机绝对路径，例如 `E:/CourseProjectC/...`。如果项目迁移到其他电脑，需要手动修改路径。

2. 模型元信息硬编码

当前输入节点名、输出节点名、输出维度、类别数等信息在代码中写死。如果之后更换不同 YOLO 模型，可能需要修改后处理代码。

3. 检测类别扩展不够完整

当前界面预留了多目标检测任务入口，但实际只接入了美团单车检测模型。后续需要引入模型配置文件或任务管理结构。

4. 训练数据规模较小

当前数据集数量有限，模型已经具备基本识别能力，但面对复杂场景仍可能出现漏检或误检。

5. 检测器资源管理仍可优化

当前 `YoloDetector` 中仍使用裸指针管理 `Ort::Session`，后续可以使用智能指针或直接成员对象提高安全性。

6. 只支持图片检测

当前系统主要支持静态图片检测，尚未支持视频检测、摄像头实时检测和批量图片检测。

9.2 后续改进方向

1. 将本机路径改为 CMake 配置变量或相对路径。
2. 自动读取 ONNX 模型输入输出信息，减少硬编码。
3. 使用配置文件管理不同模型、类别名称和颜色。
4. 增加更多不同场景、光照、角度和遮挡情况的数据。
5. 引入更多负样本，降低普通自行车、摩托车等误检概率。
6. 支持检测结果导出为图片、JSON 或 CSV。
7. 增加视频文件检测和摄像头实时检测功能。
8. 将绘制逻辑从 MainWindow 中进一步拆分，提高模块化程度。
9. 对模型进行进一步调参，例如训练更多轮数、增加数据增强或尝试更大模型。

十、实验总结

本项目从 YOLO 官方模型部署出发，逐步完成了数据采集、人工标注、模型训练、ONNX 导出、C++ 推理部署和 Qt 图形界面开发，形成了一个完整的目标检测应用闭环。

在算法层面，项目实现了目标检测系统中较关键的几个步骤，包括 Letterbox 预处理、ONNX Runtime 推理、YOLO 输出解析、坐标还原和 NMS 后处理。在工程层面，项目完成了 CMake 构建配置、Qt 界面设计、OpenCV 图像读取以及本地模型部署。在数据层面，项目通过校园实拍数据训练了一个美团单车专项模型，使系统具有明确的实际应用场景。

通过本项目，我初步理解了计算机视觉模型从训练到部署的完整流程，也体会到数据质量、标注标准、模型格式转换和工程化部署对最终系统效果的重要影响。相比只调用现成模型，本项目更强调从真实需求出发，围绕一个具体目标完成端到端实现。

当前系统仍然存在模型精度、路径配置、类别扩展和实时检测等方面的不足，但整体上看已经能够完成美团单车图片检测任务，并具备继续扩展为多目标检测平台或校园共享单车管理辅助工具的基础。